

Vision-Driven Autonomous Spacecraft Navigation

Frederik Markus¹

Amaar Quadri¹

Arvind Car¹

Brandon Lucia¹

Zachary Manchester¹

Abstract—Current orbit-determination methods rely heavily on Earth-based infrastructure like ground-based radars, GPS, and NASA’s Deep-Space Network. This paper introduces a joint orbit-and-attitude-determination method using angular velocity data from a gyroscope and one or more low-cost RGB cameras. Machine-vision algorithms are used to identify known landmarks in images captured by the spacecraft. These landmark locations are combined with gyro data and a spacecraft dynamics model in an Iterated Extended Kalman Filter (IEKF) that estimates the position, velocity, and attitude of the spacecraft, as well as the gyroscope bias. We validate the performance of the filter in simulation using real satellite imagery.

I. INTRODUCTION

Small, cost-effective satellites have created new possibilities for astronomy, Earth observation, and education. With NASA’s Artemis program aiming to reach the Moon this decade and huge interest in landing humans on Mars, there is also a strong need to develop accurate autonomous navigation methods that are less reliant on Earth-based infrastructure.

Current missions beyond Earth orbit rely primarily on Earth-based radio tracking systems like NASA’s Deep Space Network [1]. Some work [2], [3] has also been done to assess the feasibility of using Earth-based Global Navigation Satellite Systems (GNSS) such as GPS, GLONASS, BeiDou or Galileo to provide accurate position estimates on the moon. In both cases, navigation accuracy suffers due to the large distances, low Signal-to-Noise (SNR) ratios, and poor relative geometry between receiver and transmitter.

We introduce a navigation method that incorporates information from imagery captured by onboard cameras and angular velocity from a gyro. Our system can operate autonomously and completely independently from Earth-based resources. It can also be retrained easily with new imagery for bootstrapping satellite navigation systems at other celestial bodies. Our main contributions include:

- A general end-to-end visual-inertial navigation model for full state estimation onboard a spacecraft using only low-cost cameras, gyroscopes, and onboard compute.
- Experimental results that demonstrate convergence of the state estimate from initial position errors of more than 10 kilometers.
- A simulator that can generate Earth imagery for any position and attitude, using a large dataset of real and synthetic images taken by satellites, that can be used to train vision-based navigation algorithms.

¹The authors are with Carnegie Mellon University, Pittsburgh, PA 15213, USA; E-Mail: {fmarkus, aquadri, acar, blucia, zmanches}@andrew.cmu.edu

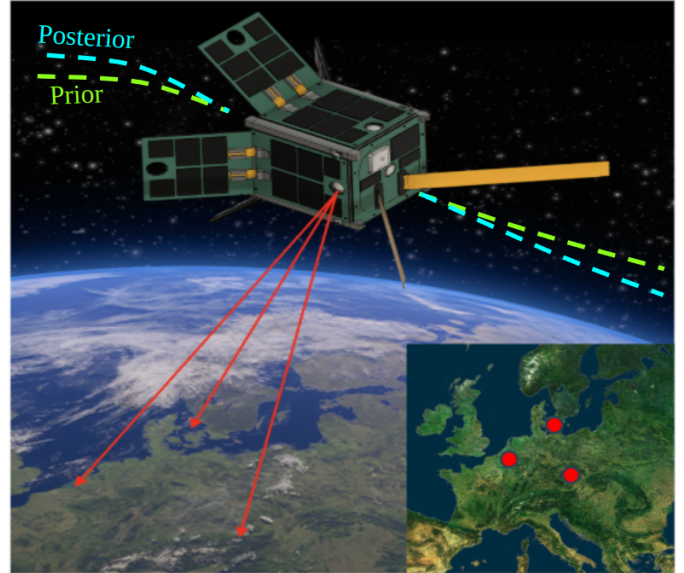


Fig. 1. The CMU ARGUS CubeSat will use visual measurements of bearings to landmarks on Earth’s surface to estimate both its orbital state and attitude.

II. BACKGROUND AND RELATED WORK

This section briefly surveys prior related work and provides a short reviews of relevant state estimation and spacecraft navigation topics.

A. Batch-Least Squares Estimation

Previous work in the field of visual-inertial navigation has aimed to solve the lost-in-space problem, where a spacecraft must localize itself without any prior state knowledge, using solely bearing measurements from an RGB camera and an inertial measurement unit (IMU) [4]. This low-cost method performed favorably against much more expensive systems, achieving position errors less than 5 km within 4 orbits 85% of the time. A significant drawback is that only the spacecraft’s position and velocity could be accurately determined, with another method assumed to provide the spacecraft’s attitude and gyro bias to high accuracy. In addition, the batch least-squares approach relies on (re-)solving a large optimization problem at every timestep.

In contrast, a recursive filter can update its state estimate incrementally, adapting to the changing conditions on Earth, where shifting weather and cloud patterns lead to constant change in the observed landscape, making the estimated parameters more resilient to measurement outliers. Working on a small CubeSat, available hardware resources are also

constrained and using a recursive filter allows for incremental updates to the state and covariance matrix. The computational overhead associated with these updates is much smaller than what would be needed for an equivalent batch optimization problem.

B. Multiplicative Extended Kalman Filter (MEKF)

For systems that solely reason about position and velocity states, the Extended Kalman Filter (EKF) often suffices [5], [6]. However, in scenarios where attitude needs to be represented through a quaternion state, this no longer works well. Using a multiplicative EKF (MEKF) [7], we can formulate the state updates to the quaternion as a group multiplication operation, rather than vector addition. This way we are able to ensure that quaternion norm constraints are maintained while also incorporating measurement updates from the gyroscope in a consistent manner. MEKFs have proven themselves to be a reliable tool to estimate attitude across a range of devices and applications [8]–[10].

C. Spacecraft Relative Navigation

Spacecraft relative navigation aims to determine the relative orbital elements of one satellite with respect to another. Previous work has explored the possibility of relative navigation using ranging-based measurements for satellite swarms with a single anchor-node satellite possessing a GPS module [11]. Vision based bearing-only methods have shown promise when navigating to a passive client spacecraft [12] as well as detecting and tracking multiple other satellites in visual imagery [13]. Various filter designs, such as the unscented Kalman filter presented in [14], can yield accurate state estimates. Nevertheless, the absolute localization problem remains unobservable without additional measurements. Our implementation applies these bearing-only ideas to landmark observations on the Earth’s surface, providing global observability.

III. SYSTEM DESIGN

This section outlines the architecture of the entire system, starting with a high-level overview, before delving into a detailed breakdown of each subsystem.

A. Overview

A block diagram of the system is shown in Figure 3. The process starts with the camera capturing an image at a resolution of 4608×2592 pixels. For the purposes of the simulation, this imagery was generated using Google’s Earth Engine API [15]. A sample image, illustrating the type of image generation that is possible, can be seen in Figure 2. The system geolocates imagery by matching image features to known landmarks on Earth. It first determines the coarse geographic region the satellite is orbiting over, then aligns ground landmarks with corresponding pixels in the image. The first of those steps, region classification, is handled by a Region Classification (RC) Network. The RC network is a trained deep-learning model that takes in a down-sampled 640×360 image and predicts which region on Earth is in

that image. Based on this output, the image is routed to a specialized Landmark Detection (LD) network, trained to identify landmarks within a particular region, mapping the corresponding pixel coordinates to location coordinates on Earth. These coordinates are used in the measurement update step of the filter.

B. Region Classifier

The RC network is trained to classify each image into a subset of 40 preselected “high-saliency” regions, narrowing the search space for landmark detection. These regions are defined using the Military Grid Reference System (MGRS), a NATO standard derived from the Universal Transverse Mercator (UTM) metric grid system [16] that divides Earth into structured grid sections. To handle cases where images span multiple regions or lack any salient features, the network is trained with multi-hot encodings. If an image does not correspond to a salient region—an area with visually distinctive features essential for geolocation—the encoding is entirely zero. This trains the network to disregard images that do not contain “useful” geographic information.

The RC network is based on the EfficientNet-B0 architecture [17], optimized for real-time inference. To adapt it for region classification, its output layer is modified with a sigmoid activation function, enabling multi-label predictions. During inference, the network assigns probability scores to each region, and those exceeding a predefined threshold determine which Landmark Detector networks will be used to identify landmarks.

C. Landmark Detector

Each LD network is trained separately for each salient region and takes an image as input, outputting a set of detected landmarks along with their pixel locations. Our model selects landmarks based on the saliency of features within a given region of interest (ROI). To identify salient features, we apply OpenCV’s saliency API [18] to Landsat 9 image data [19] for each of the 40 defined regions. Within each region, we iteratively select the top 500 most salient bounding boxes, varying in size from 5 km to 50 km.

The system’s LD network is built upon the YOLOv8 object detection model [20], trained to generate bounding boxes around landmarks in an image. To support OD, we represent both landmarks and detection bounding boxes as points by extracting their center coordinates.

D. Filter

Figure 3 provides an outline of the filter architecture. The filter estimates four three-dimensional state variables, a single four-dimensional state and a single scalar variable for a total of 17 states. These states are: the spacecraft’s position $\mathbf{r}$ in an Earth-centered inertial (ECI) frame, the spacecraft’s velocity $\mathbf{v}$, also in ECI, the spacecraft’s attitude $\mathbf{q}$, represented using a quaternion, the gyroscope’s bias $\mathbf{w}_b$, an unmodeled accelerations term $\mathbf{u}_a$ that aims to estimate the dynamics terms that are not otherwise modeled in the filter, and a drag

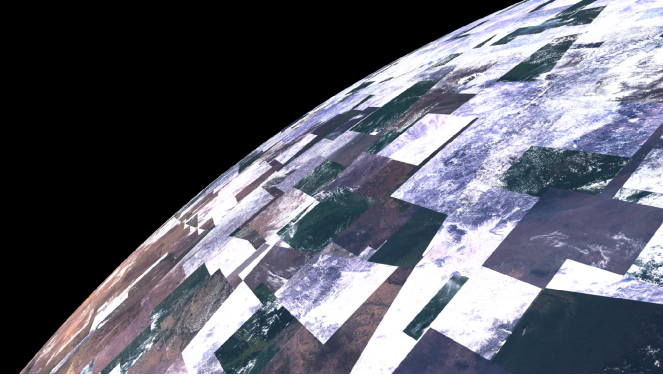


Fig. 2. A sample image illustrating the type of off-nadir Earth imagery that is used in the region classifier and the landmark detector.

estimation scalar d_{est} that is used to provide an estimate of the drag in the system.

$$\mathbf{x} = \begin{bmatrix} \mathbf{r} \\ \mathbf{v} \\ \mathbf{u}_a \\ \mathbf{q} \\ \mathbf{w}_b \\ d_{est} \end{bmatrix} \in \mathbb{R}^{17} \quad (1)$$

We can express the continuous time system dynamics of the filter as follows:

$$f_c(\mathbf{x}) = \begin{cases} \dot{\mathbf{r}} = \mathbf{v} \\ \dot{\mathbf{v}} = \frac{-\mathbf{r}GM_{\text{Earth}}}{\|\mathbf{r}\|^3} + \mathbf{u}_a + \mathbf{a}_{\text{drag-est}} + \mathbf{a}_{J2} \\ \dot{\mathbf{w}}_b = 0 \\ \dot{\mathbf{q}} = \frac{1}{2}\mathbf{q} \otimes \hat{\mathbf{w}}_{IMU} \\ \dot{\mathbf{u}}_a = 0 \\ \dot{d}_{est} = 0 \end{cases} \quad (2)$$

It is important to note the difference in acceleration dynamics between the ground-truth simulation and the filter. The filter $\dot{\mathbf{v}}$ dynamics shown above are simplified. The ground-truth simulation uses:

$$\dot{\mathbf{v}} = \frac{-\mathbf{r}GM_{\text{Earth}}}{\|\mathbf{r}\|^3} + \mathbf{a}_{\text{drag-gt}} + \mathbf{a}_{J2} + \mathbf{a}_{\text{sun}} + \mathbf{a}_{\text{moon}} \quad (3)$$

where $\mathbf{a}_{J2}$ represents the acceleration due to the J2 perturbation [21], and $\mathbf{a}_{\text{moon}}$ and $\mathbf{a}_{\text{sun}}$ account for third body accelerations due to the moon and sun respectively.

1) *Dynamic Model Compensation (DMC)*: We utilize an unmodeled acceleration term $\mathbf{u}_a$ to compensate the differences between the ground-truth dynamics and filter dynamics. Since these differences are time-correlated, simply using zero-mean Gaussian process noise does not suffice. The process of adding the $\mathbf{u}_a$ term is commonly known as Dynamic Model Compensation (DMC) and it has been shown to work for satellites in both lunar [22] and low-Earth orbits [23]. For our purposes, we assume that the unmodeled accelerations term

follows a stochastic process called a random walk. We can express this in discrete time as:

$$\mathbf{u}_a(k+1) = w(k), \quad w(k) \sim \mathcal{N}(0, \sigma_{u_a}) \quad (4)$$

2) *Drag Estimation*: For the purpose of our simulation, we calculate the atmospheric density using the Harris-Priester model [24], [25]. This takes the form:

$$\mathbf{a}_{\text{drag-gt}} = -\frac{C_d A}{2M} \rho_{HP} \mathbf{v} \|\mathbf{v}\| \quad (5)$$

where C_d represents the drag coefficient of the spacecraft, A is the surface area of the spacecraft, M the mass of the spacecraft and ρ_{HP} the atmospheric density as calculated by the Harris-Priester model. Due to the multiple orders of magnitude of drag that the spacecraft encounters throughout its time of flight [26], directly estimating the drag density is difficult. Instead we use a nominal density value and estimate a drag scaling parameter. This results in the following model for the atmospheric density based on [27]:

$$\rho_E = d_{est} \rho_0 e^{-\frac{\|\mathbf{r}\| - R_E}{H}} \quad (6)$$

ρ_0 refers to the nominal drag density value, R_E refers to the equatorial radius of Earth, and H is a reference height set at 600km, which is roughly the altitude at which we expect the spacecraft to fly. Replacing the atmospheric density in the drag equation with this model, we get the drag acceleration used by the filter as:

$$\mathbf{a}_{\text{drag-est}} = -\frac{C_d A}{2M} \rho_E \mathbf{v} \|\mathbf{v}\| \quad (7)$$

3) *Gyro Bias Estimation*: The spacecraft possesses a gyro that is used to take angular velocity measurements. These measurements are incorporated in the predictive step of the filter. The simulation uses the following gyro model [28]:

$$\mathbf{y} = \begin{bmatrix} 1 + s_1 & m_{12} & m_{13} \\ m_{21} & 1 + s_2 & m_{23} \\ m_{31} & m_{32} & 1 + s_3 \end{bmatrix} \begin{bmatrix} w_x \\ w_y \\ w_z \end{bmatrix} + \mathbf{w}_b + \mathcal{N}(0, \sigma_w) \quad (8)$$

The gyro model takes a ground-truth angular velocity $\mathbf{w}$ and produces the output $\mathbf{y}$. s_1 to s_3 refer to scaling factors present in the gyro, leading to a “stretching” of the signal. The m_{xy} terms refer to axial misalignment between axis x and y . The gyro also possesses an additive bias $\mathbf{w}_b$ that we need to consider in order to correctly estimate the attitude state of the spacecraft. The gyro bias is modeled to follow a random walk. The final term in the gyro model applies Gaussian white noise sampled from a zero mean distribution.

4) *Predictive Step*: At every timestep, the filter receives an update from the gyroscope providing a measurement of its current angular velocity, denoted as $\hat{\mathbf{w}}_{IMU}$. We then run the predictive step of the filter.

f_d represents the discretized version of the continuous time system dynamics f_c . The Jacobian term $\mathbf{A}_d$ represents the discretized version of the continuous time partial derivatives of the dynamics. We can represent the continuous time Jacobian $\mathbf{A}_c$:

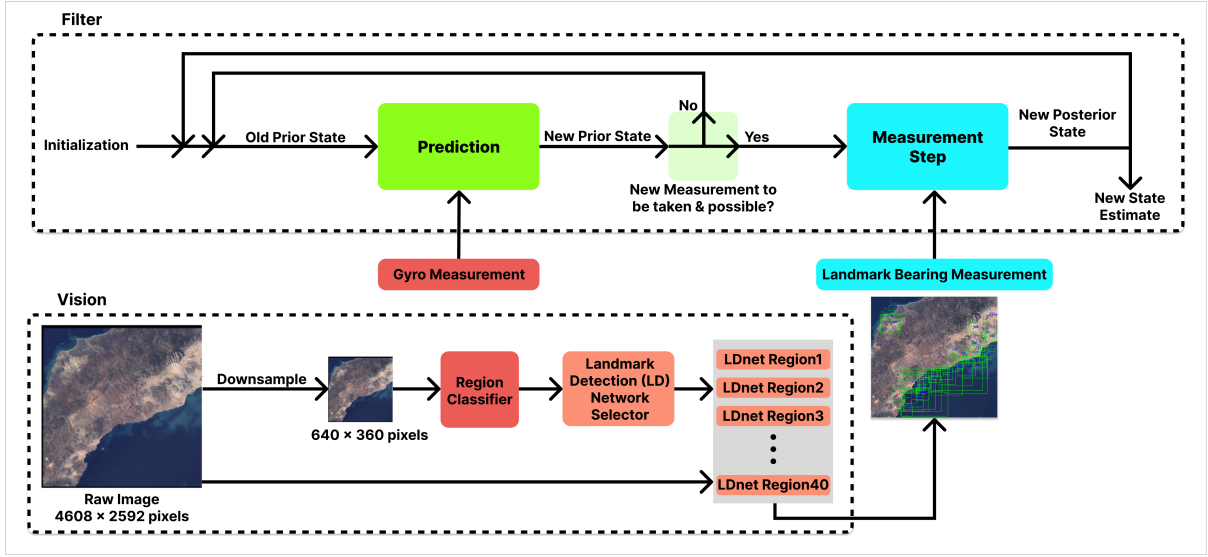


Fig. 3. Architectural Overview of the vision model and the filter. The diagram illustrates the recursive nature of the filter as new landmark bearing measurements from the vision model on the bottom are fed into the measurement step of the filter. This is then used to calculate the prediction step in the next timestep using the gyro measurements from the onboard IMU as input.

Algorithm 1 Predictive Step

Given: $\hat{\mathbf{w}}_{IMU} \in \mathbb{R}^3$, $\mathbf{P}_{k|k} \in \mathbb{R}^{17 \times 17}$, $\mathbf{x}_{k|k} \in \mathbb{R}^{17}$
 $\mathbf{x}_{k+1|k} \leftarrow f_d(\mathbf{x}_k, \hat{\mathbf{w}}_{IMU})$
 $\mathbf{A}_d \leftarrow \frac{\partial f_d}{\partial \mathbf{x}} \bigg|_{\hat{\mathbf{w}}_{IMU}, \mathbf{x}_{k|k}}$
 $\mathbf{P}_{k+1|k} \leftarrow \mathbf{A}_d \mathbf{P}_{k|k} \mathbf{A}_d^\top + \mathbf{Q}$
 return $\mathbf{x}_{k+1|k}$, $\mathbf{P}_{k+1|k}$

$$\mathbf{A}_c = \begin{bmatrix} 0_{3 \times 3} & \mathbb{I}_3 & 0_{3 \times 3} & 0_{3 \times 4} & 0_{3 \times 3} & 0_{3 \times 1} \\ \frac{\partial \dot{\mathbf{v}}}{\partial \mathbf{r}} & \frac{\partial \dot{\mathbf{v}}}{\partial \mathbf{v}} & \mathbb{I}_3 & 0_{3 \times 4} & 0_{3 \times 3} & \frac{\partial \dot{\mathbf{v}}}{\partial d_{est}} \\ 0_{3 \times 3} & 0_{3 \times 3} & 0_{3 \times 3} & 0_{3 \times 4} & 0_{3 \times 3} & 0_{3 \times 1} \\ 0_{4 \times 3} & 0_{4 \times 3} & 0_{4 \times 3} & \frac{\partial \dot{\mathbf{q}}}{\partial \mathbf{q}} & \frac{\partial \dot{\mathbf{q}}}{\partial \mathbf{w}_b} & 0_{4 \times 1} \\ 0_{3 \times 3} & 0_{3 \times 3} & 0_{3 \times 3} & 0_{3 \times 4} & 0_{3 \times 3} & 0_{3 \times 1} \\ 0_{1 \times 3} & 0_{1 \times 3} & 0_{1 \times 3} & 0_{1 \times 4} & 0_{1 \times 3} & 0_{1 \times 1} \end{bmatrix} \in \mathbb{R}^{17 \times 17} \quad (9)$$

With the matrix $\mathbf{A}_d$ calculated, we can calculate the predictive Covariance matrix $\mathbf{P}_{k+1|k}$ with $\mathbf{Q}$ representing the process covariance noise with entries corresponding to the squared units of the respective state [29]:

$$\mathbf{Q} = \text{diag}(10^{-12} \mathbb{I}_3, 10^{-16} \mathbb{I}_3, 10^{-8} \mathbb{I}_3, 10^{-12} \mathbb{I}_4, 10^{-6} \mathbb{I}_3, 10^{-6}) \quad (10)$$

5) *Measurement Step:* Due to the constrained resources on the used hardware, the measurement update is not done at every timestep but instead at regular intervals. To do a measurement update, the filter receives a measurement tuple from the vision pipeline consisting of n bearing measurements $\mathbf{z}_m$ and the associated landmark locations $\mathbf{z}_l$. With this tuple, the measurement step is taken as follows:

Algorithm 2 Measurement Step

Given: $\mathbf{P}_{k+1|k} \in \mathbb{R}^{17 \times 17}$, $\mathbf{x}_{k+1|k} \in \mathbb{R}^{17}$, num_iter $\in \mathbb{R}$, $(\mathbf{z}_m, \mathbf{z}_l) \in \mathbb{R}^{n \times 3}$, $\mathbf{t} \in \mathbb{R}$
for i = 1:num_iter **do**
if i == 0 **then**
 $\bar{\mathbf{x}}^i \leftarrow \mathbf{x}_{k+1|k}$
else
 $\bar{\mathbf{x}}^i \leftarrow \mathbf{x}_{k+1|k+1}^{i-1}$
end if
 $\mathbf{h}^i \leftarrow f_{est}(\bar{\mathbf{x}}^i, \mathbf{z}_l, \mathbf{t})$
 $\mathbf{H}^i \leftarrow \frac{\partial f_{est}}{\partial \mathbf{x}} \bigg|_{\bar{\mathbf{x}}^i, \mathbf{z}_l, \mathbf{t}}$
 $\mathbf{K}^i \leftarrow \mathbf{P}_{k+1|k} \mathbf{H}^i (\mathbf{H}^i \mathbf{P}_{k+1|k} \mathbf{H}^{i\top} + \mathbf{R})^{-1}$
 $\mathbf{x}_{k+1|k+1, q}^i \leftarrow \bar{\mathbf{x}}^i + \mathbf{K}^i (\mathbf{z}_m - \mathbf{h}^i)$
 $\mathbf{x}_{k+1|k+1, q}^i \leftarrow \bar{\mathbf{x}}^i * \mathbf{K}^i (\mathbf{z}_m - \mathbf{h}^i)$
 $\mathbf{P}_{k+1|k+1} \leftarrow (\mathbb{I}_{16} - \mathbf{K}^i \mathbf{H}^i) \mathbf{P}_{k+1|k} (\mathbb{I}_{16} - \mathbf{K}^i \mathbf{H}^i)^\top + \mathbf{K}^i \mathbf{R} \mathbf{K}^{i\top}$
end for
 return $\mathbf{x}_{k+1|k+1}$, $\mathbf{P}_{k+1|k+1}$

It must be noted that the filter does an iterative step in the measurement update. By running the measurement update multiple times, we are able to improve the performance of the filter significantly. It is also important to recognize the difference in state update between the non-quaternion states $\mathbf{x}_{k+1|k+1, q}^i$ and the quaternion state $\mathbf{x}_{k+1|k+1, q}^i$. This is where the multiplicative nature of the quaternion state becomes clear which defines the multiplicative part of our filter. The measurement noise covariance $\mathbf{R}$ scales in each

measurement update with the number of measurements taken:

$$\mathbf{R} = \begin{bmatrix} 10^{-2} & 0 & 0 & 0 & 0 \\ 0 & 10^{-2} & 0 & 0 & 0 \\ 0 & 0 & 10^{-2} & 0 & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & 0 & 10^{-2} \end{bmatrix} \in \mathbb{R}^{3n \times 3n} \quad (11)$$

The estimation function f_{est} is central to the measurement step. Here we combine the landmark location measurements with the current state estimate (both provided in ECI) and the spacecraft's clock to return an estimate of the expected bearing measurements:

Algorithm 3 f_{est}

Given: $\mathbf{x} \in \mathbb{R}^{17}$, $\mathbf{z}_1 \in \mathbb{R}^{n \times 3}$, $\mathbf{t} \in \mathbb{R}$
 $ECI \mathbf{R}_{body} \leftarrow f_{qtoR}(\mathbf{q})$
 $ECEF \mathbf{R}_{ECI} \leftarrow f_{ECItoECEF}(\mathbf{t})$
 $ECEF \mathbf{R}_{body} \leftarrow ECEF \mathbf{R}_{ECI} * ECI \mathbf{R}_{body}$
 $\mathbf{z}_{1ECEF} \leftarrow ECEF \mathbf{R}_{ECI} * \mathbf{z}_1$
 $\mathbf{x}_{rECEF} \leftarrow ECEF \mathbf{R}_{ECI} * \mathbf{x}_r$
 $\mathbf{c}_{ECEF} \leftarrow f_{cam}(\mathbf{x}_{rECEF}, ECEF \mathbf{R}_{body})$
 $\mathbf{v}_{ECEF} \leftarrow (\mathbf{z}_{1ECEF} - \mathbf{c}_{ECEF}) / \|\mathbf{z}_{1ECEF} - \mathbf{c}_{ECEF}\|$
 $\mathbf{v}_{body} \leftarrow ECEF \mathbf{R}_{body}^\top * \mathbf{v}_{ECEF}$
 return $\mathbf{v}_{body}$

f_{cam} defines the camera association function. This function is necessary to correctly identify the camera that a particular measurement came from and provide the associated rotation matrix that is needed to rotate the measurement from a certain camera frame into the body frame. As multiple cameras may be able to see the Earth at once, distinguishing measurements by their camera origin is critical to estimate the correct attitude of the spacecraft. The time term $\mathbf{t}$ is necessary to correctly map landmarks on Earth from an Earth-Centered-Earth-Fixed (ECEF) to an ECI frame. In our implementation this is done using the Python library Brahe [30].

6) *Initialization:* In order to initialize the filter, the spacecraft requires a rough knowledge of its state. This means that it needs to solve the lost-in-space problem as shown in [4]. It also requires an initial guess on its attitude and the gyro bias, which can be solved using a batch least squares implementation. For the unmodeled acceleration state term it usually suffices to initialize with an approximate calculation of the magnitude of the unmodeled accelerations, as it is comparatively small when seen in contrast to the other states and the properties of the filter will cause it to naturally settle into its steady state.

IV. EVALUATION

In this section we evaluate the performance of the vision model and filter through a set of Monte Carlo simulations. We initialize our spacecraft in a fashion that is representative of a CubeSat in a 600 km altitude sun-synchronous orbit.

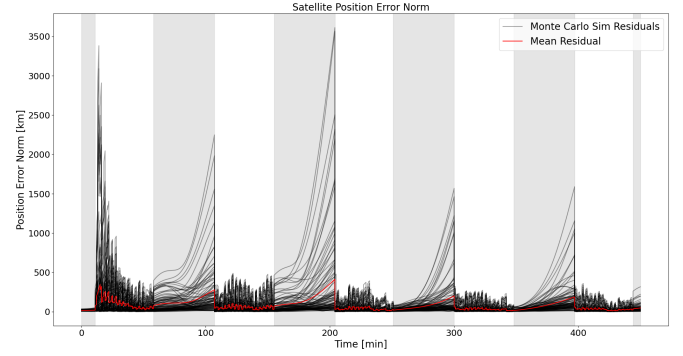


Fig. 4. Monte Carlo simulation of 100 trials starting from the same ground truth position but initialized with a different filter error. Each trial runs for a total of 5 orbits. The red line indicates the mean position residual. The sections in gray show the night sections of the orbit.

A. Position Error

Figure 4 illustrates the key result of the filter. Over several orbits, the position error improves markedly. Note the divergence of the position error that occurs periodically in every orbit at night, shaded in gray. As our models are not trained to work with night-time imagery, our satellite is essentially “blind” and using dead-reckoning based on its belief state and known spacecraft dynamics. This leads to a significant increase in state error that reduces with further orbits as the filter stabilizes. The line in red indicates the mean filter position error over all 100 Monte Carlo trials.

B. Consistency of the Filter

We are interested in assessing the consistency of the filter. To this end we provide the 3σ bounded area for all position axes. If the position residual lies outside of the 3σ bound, the filter is over-confident in its estimate of the state. Figure 5 illustrates that the filter is consistent almost immediately after initialization. Furthermore, with more orbits, the position residual reduces further during the phases where no measurements are present. We were also able to show experimentally that using more measurements (by using lower saliency features or a larger fraction of measurements) we were able to drive down the 3σ -bound significantly, resulting in improved confidence, even in situations where no measurements were available.

C. Gyro Bias Drift

As with all gyros, there is a natural bias drift associated with the system. Figure 6 illustrates the convergence of the bias error over time. The re-introduction of measurements once it becomes daytime induce oscillations in the bias error that settle after sufficient time has passed. Note the apparent constant offset of the z-axis that results not from the filter estimation but rather the simulation regimen used, causing weaker observability.

V. CONCLUSIONS & OUTLOOK

Our system is, to our knowledge, the first that performs full satellite orbital and attitude state estimation using only

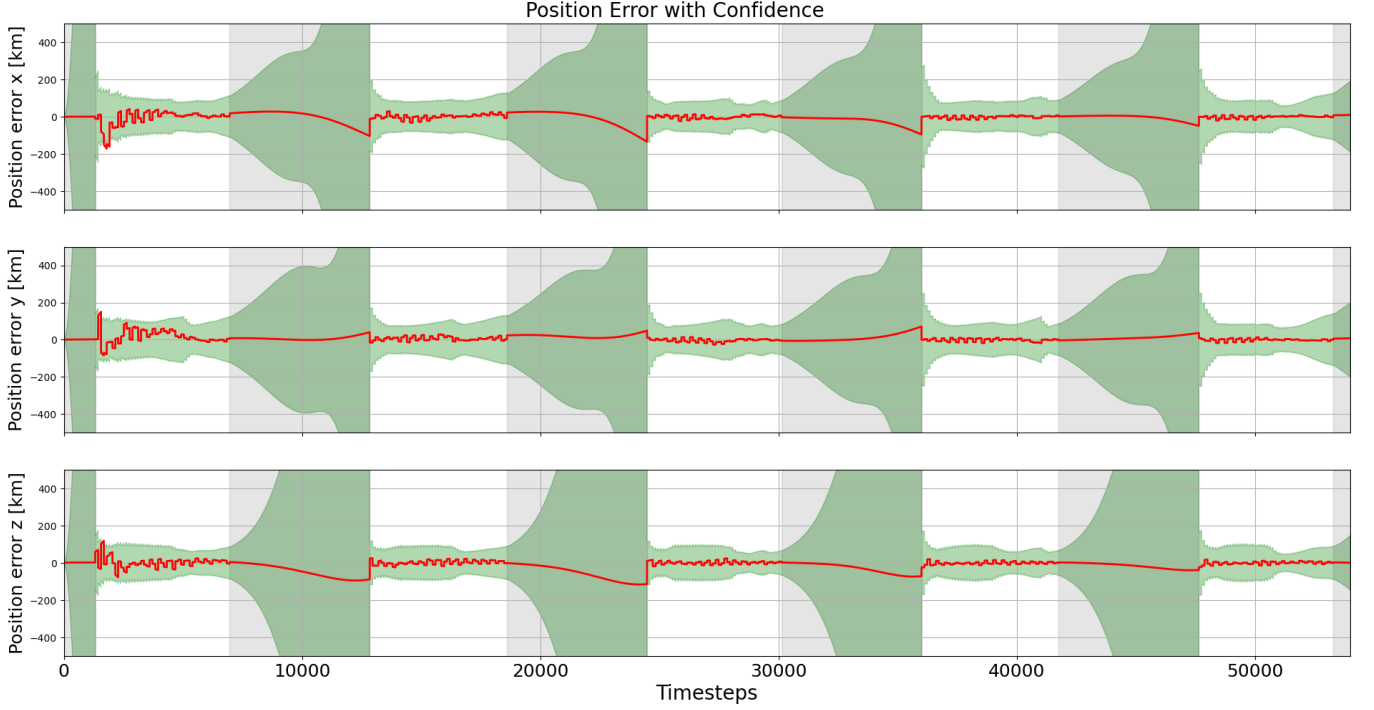


Fig. 5. Mean position residuals over 100 Monte Carlo trials and mean 3σ -bounded area in green for all three position dimensions.

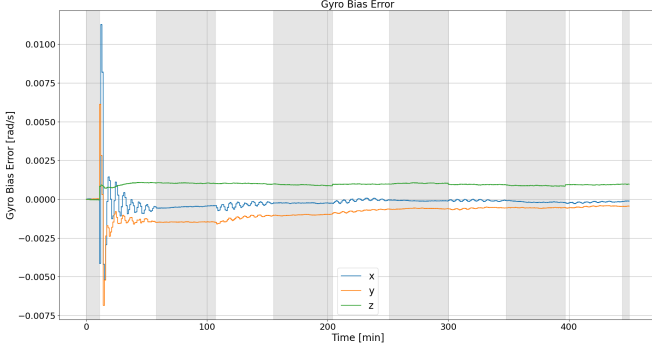


Fig. 6. Averaged Gyro Bias Error of 100 Monte Carlo trials over 5 orbits. Note that the apparent constant offset of the z-axis results not from the filter but rather the simulation regimen used, causing weaker observability.

measurements from onboard cameras and a gyro. We achieve convergence to less than 10 km position error in 4 orbits on all attempted Monte Carlo trials. The night-time flare-up in error does continue to occur but becomes less and less pronounced. Besides the lack of exogenous signals required, the greatest benefit of the system is the simple redeployment ability: It only requires satellite imagery to retrain the vision networks to re-purpose our system from being used in Earth orbit to being used in orbit around any other celestial body, like the moon or Mars.

The most straightforward improvement that can be done is to include nighttime imagery in the training data. NASA’s black marble imagery [31] could be a good starting point

for this. With this we should be able to avoid the temporary divergence in state error that occurs and reach an acceptable level of convergence faster.

Other possible improvements relate to the scheduling of the measurements: The current implementation uses a set measurement rate of one measurement every 60 seconds. This could be modified to consider variable rates based on, for example, the covariance trace. One could set a threshold limit on the covariance trace beyond which a measurement is triggered. Another possible implementation could use a Reinforcement Learning agent to decide, based on the current states, covariance, and saliency of the landmarks being observed, whether to trigger a measurement or not.

There are also questions related to the number of measurement iterations to be done for each measurement step. The current implementation uses 5 measurement iterations, but this number was solely chosen for the fact that it provided a good trade-off between computation time and convergence benefits.

One exciting prospect is the deployment to hardware. In the near term, the goal is to deploy the entire end-to-end system to a Nvidia Jetson Orion that is to be flown to space as part of the ARGUS mission in late 2025.

ACKNOWLEDGMENT

The authors would like to thank all those that helped in the course of this research, including CMU’s Design-Build-Fly Laboratory course for their refinement of concepts in this paper.

REFERENCES

- [1] D. J. Mudgway, *Uplink-downlink: a history of the nasa deep space network, 1957-1997*. National Aeronautics and Space Administration, 2001, vol. 4227.
- [2] V. Capuano, C. Botteron, J. Leclère, J. Tian, Y. Wang, and P.-A. Farine, "Feasibility study of gnss as navigation system to reach the moon," *Acta Astronautica*, vol. 116, pp. 186–201, 2015. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0094576515002520>
- [3] G. B. Palmerini, M. Sabatini, and G. Perrotta, "En route to the moon using gnss signals," *Acta Astronautica*, vol. 64, no. 4, pp. 467–483, 2009. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0094576508003007>
- [4] K. McCleary, S. Gurumurthy, P. R. Fisch, S. Tayal, Z. Manchester, and B. Lucia, "Vinsat: Solving the lost-in-space problem with visual-inertial navigation," in *2024 IEEE International Conference on Robotics and Automation (ICRA)*, 2024, pp. 11 774–11 780.
- [5] L. Loron and G. Laliberte, "Application of the extended kalman filter to parameters estimation of induction motors," in *1993 Fifth European Conference on Power Electronics and Applications*, 1993, pp. 85–90 vol.5.
- [6] Z. Huang, P. Du, D. Kosterev, and B. Yang, "Application of extended kalman filter techniques for dynamic model parameter calibration," in *2009 IEEE Power & Energy Society General Meeting*, 2009, pp. 1–8.
- [7] E. Lefferts, F. Markley, and M. Shuster, "Kalman filtering for spacecraft attitude estimation," *Journal of Guidance, Control, and Dynamics*, vol. 5, no. 5, pp. 417–429, 1982. [Online]. Available: <https://doi.org/10.2514/3.56190>
- [8] P. Martin and E. Salaün, *Generalized Multiplicative Extended Kalman Filter for Aided Attitude and Heading Reference System*, 2012. [Online]. Available: <https://arc.aiaa.org/doi/abs/10.2514/6.2010-8300>
- [9] R. C. Leishman and T. W. McLain, "Multiplicative extended kalman filter for relative rotorcraft navigation," *Journal of Aerospace Information Systems*, vol. 12, no. 12, pp. 728–744, 2015. [Online]. Available: <https://doi.org/10.2514/1.1010236>
- [10] Y. Kim, S. Lim, and H. Bang, "Attitude estimation for unmanned aircraft using a multiplicative extended kalman filter," *Measurement*, vol. 1, p. 7, 2013.
- [11] M. Holliday, R. Ticknor, J. Stupl, R. Hunter, P. Fisch, I. S. Sow, J. Willis, and Z. Manchester, "The py4 mission: A low-cost demonstration of cubesat formation-flying technologies," 2024.
- [12] S. D'Amico, J.-S. Ardaens, G. Gaias, H. Benninghoff, B. Schlepp, and J. L. Jørgensen, "Noncooperative rendezvous using angles-only optical navigation: System design and flight results," *Journal of Guidance, Control, and Dynamics*, vol. 36, no. 6, pp. 1576–1595, 2013. [Online]. Available: <https://doi.org/10.2514/1.59236>
- [13] J. Kruger, S. S. Hwang, and S. D'Amico, "Starling formation-flying optical experiment: Initial operations and flight results," 2024. [Online]. Available: <https://arxiv.org/abs/2406.06748>
- [14] J. Sullivan and S. D'Amico, "Nonlinear kalman filtering for improved angles-only navigation using relative orbital elements," *Journal of Guidance, Control, and Dynamics*, vol. 40, no. 9, pp. 2183–2200, 2017. [Online]. Available: <https://doi.org/10.2514/1.G002719>
- [15] R. Moore and M. Hansen, "Google earth engine: a new cloud-computing platform for global-scale earth observation data and analysis," in *AGU Fall Meeting Abstracts*, vol. 2011, 2011, pp. IN43C–02.
- [16] P. H. Stott, "The utm grid reference system," *IA. The Journal of the Society for Industrial Archeology*, vol. 3, no. 1, pp. 1–14, 1977. [Online]. Available: <http://www.jstor.org/stable/40967933>
- [17] V.-T. Hoang and K.-H. Jo, "Practical analysis on architecture of efficientnet," in *2021 14th International Conference on Human System Interaction (HSI)*. IEEE, 2021, pp. 1–4.
- [18] L. Yu, W. Sun, H. Wang, Q. Wang, and C. Liu, "The design of single moving object detection and recognition system based on opencv," in *2018 IEEE International Conference on Mechatronics and Automation (ICMA)*, 2018, pp. 1163–1168.
- [19] J. G. Masek, M. A. Wulder, B. Markham, J. McCorkel, C. J. Crawford, J. Storey, and D. T. Jenstrom, "Landsat 9: Empowering open science and applications through continuity," *Remote Sensing of Environment*, vol. 248, p. 111968, 2020. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0034425720303382>
- [20] M. Sohan, T. Sai Ram, and C. V. Rami Reddy, "A review on yolov8 and its advancements," in *International Conference on Data Intelligence and Cognitive Informatics*. Springer, 2024, pp. 529–545.
- [21] J. KECHICHIAN and T. KELLY, *Analytic solution of perturbed motion in near-circular orbit due to J2, J3 earth zonal harmonics in rotating and inertial Cartesian reference frames*, 1989. [Online]. Available: <https://arc.aiaa.org/doi/abs/10.2514/6.1989-352>
- [22] B. Tapley and B. Schutz, "Estimation of unmodeled forces on a lunar satellite," *Celestial mechanics*, vol. 12, no. 4, pp. 409–424, 1975.
- [23] K. A. Myers and B. D. Tapley, "Dynamical model compensation for near-earth satellite orbit determination," *AIAA Journal*, vol. 13, no. 3, pp. 343–349, 1975.
- [24] D. L. Dowd and B. Tapley, "Density models for the upper atmosphere," *Celestial mechanics*, vol. 20, no. 3, pp. 271–295, 1979.
- [25] I. Harris and W. Priester, "The upper atmosphere in the range from 120 to 800 km," Tech. Rep., 1964.
- [26] W. J. Larson and J. R. Wertz, "Space mission analysis and design," 1999.
- [27] O. Montenbruck, E. Gill, and F. Lutze, "Satellite orbits: models, methods, and applications," *Appl. Mech. Rev.*, vol. 55, no. 2, pp. B27–B28, 2002.
- [28] M. Kirkko-Jaakkola, J. Collin, and J. Takala, "Bias prediction for mems gyroscopes," *IEEE Sensors Journal*, vol. 12, no. 6, pp. 2157–2163, 2012.
- [29] Y. Ou, H. Zhang, and J. Xing, "Autonomous orbit determination and observability analysis for formation satellites," in *2016 35th Chinese Control Conference (CCC)*, 2016, pp. 5294–5300.
- [30] D. Eddy, "Brahe documentation," <https://duncaneddy.github.io/brahe>, 2025, accessed: 2025-03-14.
- [31] M. O. Román, Z. Wang, Q. Sun, V. Kalb, S. D. Miller, A. Molthan, L. Schultz, J. Bell, E. C. Stokes, B. Pandey *et al.*, "Nasa's black marble nighttime lights product suite," *Remote Sensing of Environment*, vol. 210, pp. 113–143, 2018.